



POLITECNICO
MILANO 1863

Software Engineering for Automation
A.Y. 2025–2026

Design Document

**Search-Based Test Generation for Autonomous UAV Obstacle
Avoidance Using a $(1 + 1)$ -Evolution Strategy**

Authors: Roberto Capone
Guglielmo Cattaneo

Instructor: Prof. Matteo Camilli

Version: 1.0

Release date: 17/07/2026

Revision History

Date	Version	Authors	Description
17/07/2026	1.0	Roberto Capone; Guglielmo Cattaneo	Initial complete release aligned with the final implementation contained in the project repository and Requirements Analysis and Specification version 1.0.

Contents

Revision History	1
1 Introduction	3
1.1 Purpose	3
1.2 Project Goals	3
1.3 Definitions, Acronyms, and Abbreviations	3
1.4 Document Structure	4
2 Architectural Design	5
2.1 Architectural Drivers	5
2.2 Architecture Overview	5
2.3 Component View	6
2.3.1 Command-Line and Logging Component	6
2.3.2 Configuration Component	6
2.3.3 Candidate Model Component	7
2.3.4 Geometry and Scoring Component	7
2.3.5 Simulation Adapter Component	7
2.3.6 Evolutionary Orchestration Component	8
2.3.7 Random Baseline Component	8
2.3.8 Plotting Component	8
2.3.9 Launcher and Deployment Wrapper	8
2.4 Module Dependency View	9
2.5 Class View	10
2.6 Data Design	10
2.6.1 Obstacle Genome	10
2.6.2 Checkpoint Schema	10
2.6.3 Persistent Output Layout	11
2.7 Runtime View	11
2.7.1 Fresh Evolutionary Run	12
2.7.2 Detailed Execution Flow	14
2.7.3 Crash and Resume	17
2.7.4 Final Delivery Construction	18
2.8 Algorithm Design	18
2.8.1 Search Fitness	18
2.8.2 Initialisation	18
2.8.3 Gaussian Mutation	18
2.8.4 Mutation-Scale Adaptation	19
2.8.5 Trap-Gene Operator	19
2.8.6 Stagnation and Restart	19
2.8.7 Failure Point and Delivery Score	19
2.8.8 Trajectory Representation and DTW	20

2.8.9	Archive Policy	20
2.9	Error Handling and Fault Tolerance	21
2.10	Deployment View	22
2.11	Architectural Styles and Patterns	22
2.11.1	Modular / Layered Organisation	22
2.11.2	Strategy Variation through Inheritance	22
2.11.3	Adapter	22
2.11.4	Checkpoint / Memento	22
2.11.5	Repository-Like Failure Store	23
2.11.6	Functional Core, Imperative Shell	23
2.12	Design Decisions and Trade-Offs	23
3	User Interface Design	24
3.1	Interaction Model	24
3.2	Primary Commands	24
3.2.1	Reference Launcher	24
3.2.2	Direct Generator Invocation	24
3.2.3	Self-Contained Container Build	24
3.3	Environment Configuration	24
3.4	Console Feedback	25
3.5	Output Interface	25
4	Requirements Traceability	26
4.1	Traceability Method	26
5	Implementation, Integration, and Test Plan	29
5.1	Implementation Order	29
5.2	Integration Strategy	29
5.3	Test Levels	30
5.3.1	Unit Tests	30
5.3.2	Integration Tests	30
5.3.3	System Tests	30
5.3.4	Regression Tests	31
5.4	Test Oracles	31
6	References	32
A	Implementation File Catalogue	33
B	Default Parameter Catalogue	34

Introduction

1.1 Purpose

This document defines the architecture and detailed design of the UAV Test Generator. It translates the functional and non-functional requirements stated in the Requirements Analysis and Specification Document into components, classes, interfaces, algorithms, persistent data structures, runtime interactions, and integration decisions.

The design describes the final implementation contained in the project repository. It therefore serves both as an architectural specification and as a traceability reference for reviewing the submitted source code.

1.2 Project Goals

The system is designed to generate effective and diverse failing test cases for PX4-Avoidance under a finite simulation budget. The principal design goals are:

- preserve a simple competition-compatible command-line interface;
- separate deterministic search logic from expensive simulator interaction;
- enforce obstacle validity before simulation;
- use continuous failure severity to guide a $(1 + 1)$ -Evolution Strategy;
- preserve diverse representative failures while retaining a complete persistent history;
- tolerate simulator non-determinism, timeouts, and process crashes;
- support reproducible experiments and a fair random-search baseline;
- produce inspectable and evaluator-ready output artifacts.

1.3 Definitions, Acronyms, and Abbreviations

The definitions from the Requirements Analysis and Specification Document apply. The following design-specific terms are used:

Term	Definition
Pending candidate	Genes written to the checkpoint immediately before simulation so that an interrupted test can be repeated exactly.
Route corridor	Subset of the nominal trajectory used to guide obstacle placement and tune positional mutation scales.

Term	Definition
Trajectory basin	Set of failures whose resampled trajectories are closer than the configured DTW threshold.
Trap gene	Additional obstacle targeted at the parent’s observed escape gap and oriented across the local trajectory direction.
Delivery archive	Bounded in-memory list of representative failures used to construct the final suite; distinct from the complete persistent <code>all_fails</code> history.
Variation point	Method deliberately overridden by <code>RandomGenerator</code> to replace evolutionary offspring creation with independent sampling.

1.4 Document Structure

Chapter 2 presents the architectural design, including component, module, class, data, deployment, runtime, algorithmic, and pattern views. Chapter 3 specifies the command-line user interface and persistent output interface. Chapter 4 maps requirements to design elements and implementation files. Chapter 5 defines the implementation, integration, and test plan. Chapter 6 lists references.

Architectural Design

2.1 Architectural Drivers

The architecture is shaped by the following high-impact requirements:

1. **Expensive and unstable execution.** Each simulator call may take several minutes, time out, or terminate the process; search state must therefore be persistent and simulation concerns isolated.
2. **Strict global budget.** Every attempt must be accounted for before execution and no retry may bypass the limit.
3. **External infrastructure.** PX4, Gazebo, and Aerialist are external dependencies accessed through a narrow wrapper.
4. **Validity constraints.** Rotated boxes must fit within a legal region and must not overlap.
5. **Multiple objectives with different roles.** Minimum distance guides the search, whereas simplicity, execution time, and diversity influence archive and delivery decisions.
6. **Experimental comparability.** The evolutionary and random generators must share all infrastructure except the search mechanism.
7. **Inspectable artifacts.** Every failure must be persistently represented as YAML and, when available, ULog and plot files.

2.2 Architecture Overview

The system follows a modular pipeline architecture. The command-line layer selects and starts a generator. The generator coordinates mission handling, candidate production, execution, measurement, search-state transitions, archiving, checkpointing, and delivery. Deterministic geometry and scoring functions are isolated in a functional utility module. A small data model represents candidates independently of Aerialist objects. A test-case adapter encapsulates the external simulation API.

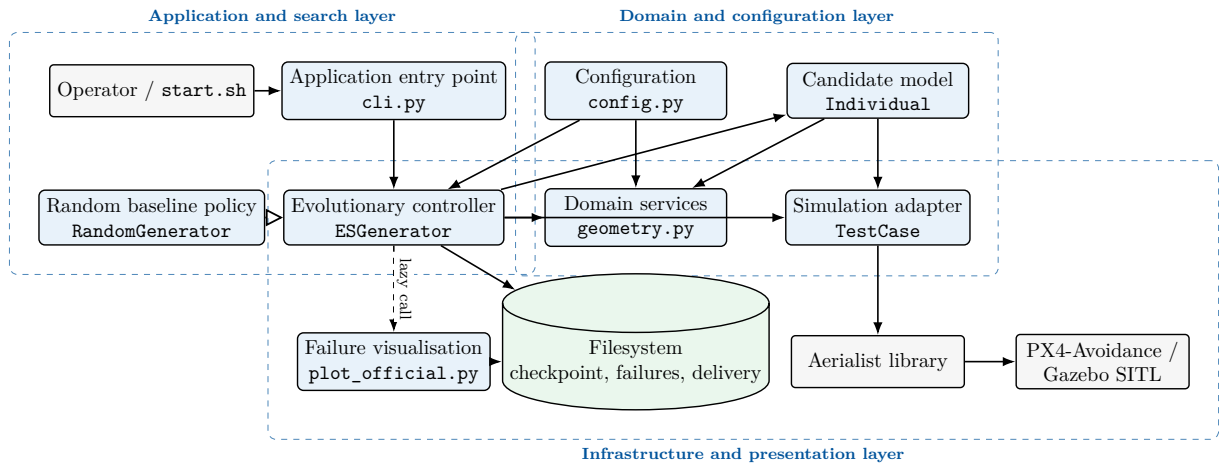


Figure 2.1: Logical component view with explicit application, domain, and infrastructure responsibilities.

2.3 Component View

2.3.1 Command-Line and Logging Component

Implementation: `cli.py`.

Responsibilities:

- parse the `generate` operation, mission path, and budget;
- select `Esgenerator` or `RandomGenerator` through the `GENERATOR` environment variable;
- configure persistent and console logging;
- create the campaign output directory;
- invoke `generate()` and `write_final_selection()`;
- convert uncaught exceptions into a logged non-zero process exit.

Provided interface:

```
python3 cli.py generate <case-study.yaml> <budget>
```

2.3.2 Configuration Component

Implementation: `config.py`.

The configuration component centralises official bounds and environment-configurable parameters. Values are materialised at module import time. It also initialises Python’s pseudo-random generator when `SEED` is present.

The principal parameter groups are:

- legal obstacle dimensions and placement bounds;
- simulation timeout, retry, and repeated-evaluation parameters;
- adaptive route sampling and destination-completion parameters;
- mutation scale and stagnation parameters;
- archive size, spatial buckets, IoU, and DTW diversity parameters;
- trap-gene and intensification parameters.

2.3.3 Candidate Model Component

Implementation: `models.py`.

`Individual` is the central domain object. It owns only simple values or temporary references and therefore separates the generator’s search state from the Aerialist execution model.

Attribute	Type	Purpose
<code>genes</code>	list of lists	One to three genes $[l, w, h, x, y, r]$.
<code>fitness</code>	optional float	Search objective; minimum distance.
<code>min_distance</code>	float	Most severe observed distance.
<code>completed</code>	bool	Whether the destination was reached with sufficient frequency.
<code>test_case</code>	optional <code>TestCase</code>	In-memory best execution artifact; not checkpointed.
<code>fail_base</code>	optional string	Relative base path of persistent failure files.
<code>avg_point</code>	optional float	Mean competition point across executions.
<code>avg_flight_min</code>	optional float	Mean duration used by delivery ranking.
<code>traj</code>	optional list	Average resampled trajectory used for DTW.

The functions `_ind_to_dict()` and `_ind_from_dict()` define an explicit serialisation boundary. Aerialist objects are excluded, infinity is represented by `None`, and backward-compatible field fallbacks are supported.

2.3.4 Geometry and Scoring Component

Implementation: `geometry.py`.

This module contains pure or mostly pure functions for:

- conversion from genes to Aerialist obstacles;
- rotated-box corner computation and overlap testing through the Separating Axis Theorem;
- rotation-aware fitting within legal bounds;
- route loading, destination detection, and destination reachability;
- random sampling, Gaussian mutation, and route-dependent step tuning;
- official point and ranking formulas;
- trajectory resampling and Dynamic Time Warping;
- non-overlapping obstacle generation, targeted trap-gene generation, and overlap repair.

Two module-level values, `_ROUTE_XY` and `_DESTINATION`, represent campaign-specific route state. `GENE_STEP` is mutable because its position entries are tuned to the loaded route.

2.3.5 Simulation Adapter Component

Implementation: `testcase.py`.

`TestCase` acts as an adapter around Aerialist. Its constructor deep-copies the immutable case study and replaces only the obstacle list. `execute()` selects the configured Aerialist agent, runs the test, and exposes the first returned trajectory and log file. `get_distances()` delegates obstacle-distance measurement to the trajectory API. `save_yaml()` serialises the enriched test description.

This component prevents the search controller from depending directly on local, Docker, or Kubernetes agent classes.

2.3.6 Evolutionary Orchestration Component

Implementation: `es_generator.py`.

`ESGenerator` is the application service and state machine of the system. Its responsibilities are grouped as follows:

- **evaluation:** retries, timeout, budget accounting, repeated executions, aggregation, completion assessment, and trajectory extraction;
- **route bootstrap:** obstacle-free simulation when no nominal route exists;
- **checkpointing:** atomic save and restoration of serialisable state;
- **search:** initial seeding, $(1 + 1)$ selection, mutation-scale adaptation, stagnation, restart, and intensification;
- **archive:** persistent failure dump, trajectory deduplication, bounded representative set, and replacement;
- **delivery:** score ordering, final DTW separation, idempotent copying, and stale-rank cleanup.

2.3.7 Random Baseline Component

Implementation: `random_generator.py`.

`RandomGenerator` inherits `ESGenerator` and overrides two variation methods:

- `_make_child()`, which returns an independently sampled one-obstacle individual;
- `_restart_individual()`, which also returns an independent sample.

Evaluation, validity, checkpointing, archive, delivery, and output formatting remain unchanged. This implements a controlled variation point: observed differences are attributable to the search policy rather than to different infrastructure.

2.3.8 Plotting Component

Implementation: `plot_official.py`.

The plotter reads ULog data through `pyulog` when available and falls back to Aerialist trajectory extraction inside the container. It creates a composite figure with x , y , z , and yaw over time and a top-down map containing rotated obstacle rectangles and trajectory start/end markers. Plotting is deliberately non-critical: an exception is logged by the generator and does not terminate search.

2.3.9 Launcher and Deployment Wrapper

Implementation: `start.sh` and `Dockerfile`.

`start.sh` constructs a mission-specific output directory, cleans obsolete containers, mounts the source and output directories, copies generator files into the Aerialist working directory, starts one container for the whole budget, and restarts it after non-zero termination. The same output directory is reused so that the generator can restore its checkpoint. The wrapper also restores host file ownership.

The `Dockerfile` supports a self-contained image build by extending the Aerialist image, installing declared dependencies, copying the generator, creating output directories, and defining the default execution agent and PX4-Avoidance resources.

2.4 Module Dependency View

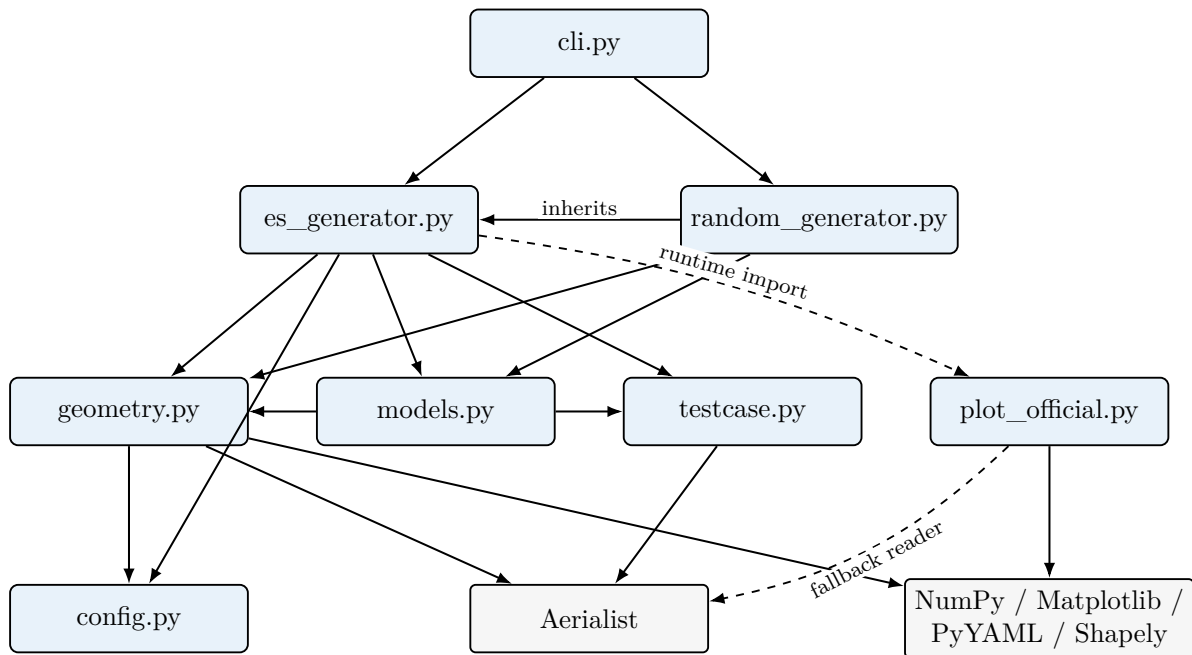


Figure 2.2: Static Python-module dependencies. Dashed edges denote imports performed only on a runtime path.

2.5 Class View

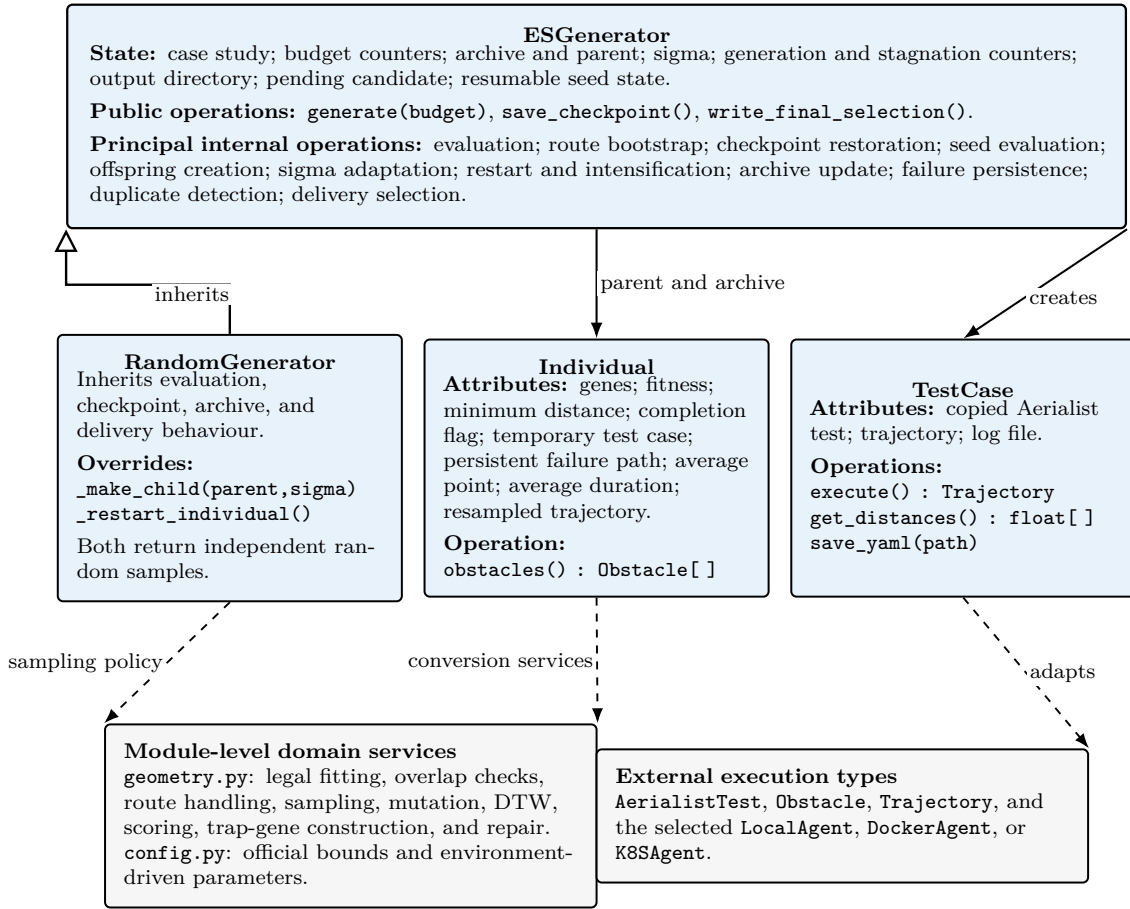


Figure 2.3: Principal class model in portrait layout, synchronized with `docs/uml/class_diagram.mmd`. Utility functions and constants remain module-level services rather than artificial classes.

2.6 Data Design

2.6.1 Obstacle Genome

A candidate genome is a list

$$G = [g_1, \dots, g_n], \quad 1 \leq n \leq 3,$$

where

$$g_i = [l_i, w_i, h_i, x_i, y_i, r_i].$$

The vertical position is not stored because it is fixed to $z_i = 0$. Gene-to-obstacle conversion occurs only when the candidate is prepared for execution or geometric comparison.

2.6.2 Checkpoint Schema

The checkpoint is a JSON object containing:

- budget and run metadata: `budget_used`, `total_budget`, and seed;
- search state: `sigma`, generation, stagnation, success history;

- crash state: pending genes;
- initialisation state: seeding flag, remaining seeds, and best seed;
- parent and serialised archive individuals;
- route and destination state.

The write protocol is:

1. serialise to `checkpoint.json.tmp`;
2. close the temporary file;
3. atomically replace `checkpoint.json` using `os.replace()`.

This prevents a process failure during writing from destroying the previous valid snapshot.

2.6.3 Persistent Output Layout

```
generated_tests/<mission>-<generator>[-sSEED]-<date>/
|-- checkpoint.json
|-- debug.txt
|-- all_fails/
|   |-- fail_d<distance>_obs<count>_run<budget>.yaml
|   |-- fail_d<distance>_obs<count>_run<budget>.ulg
|   '-- fail_d<distance>_obs<count>_run<budget>_plot.png
'-- consegna/
    |-- rank01_d<distance>_obs<count>.yaml
    |-- rank01_d<distance>_obs<count>.ulg
    '-- rank01_d<distance>_obs<count>_plot.png
```

The complete failure history is append-only from the perspective of delivery construction. Delivery files are copies; they may be rebuilt or stale ranks removed without altering `all_fails`.

2.7 Runtime View

The class, sequence, and execution-flow figures are synchronized with the repository-maintained Mermaid sources in `Deliverable_3_Implementation/docs/uml/`. The TikZ versions below are used to keep this document self-contained and printable.

2.7.1 Fresh Evolutionary Run

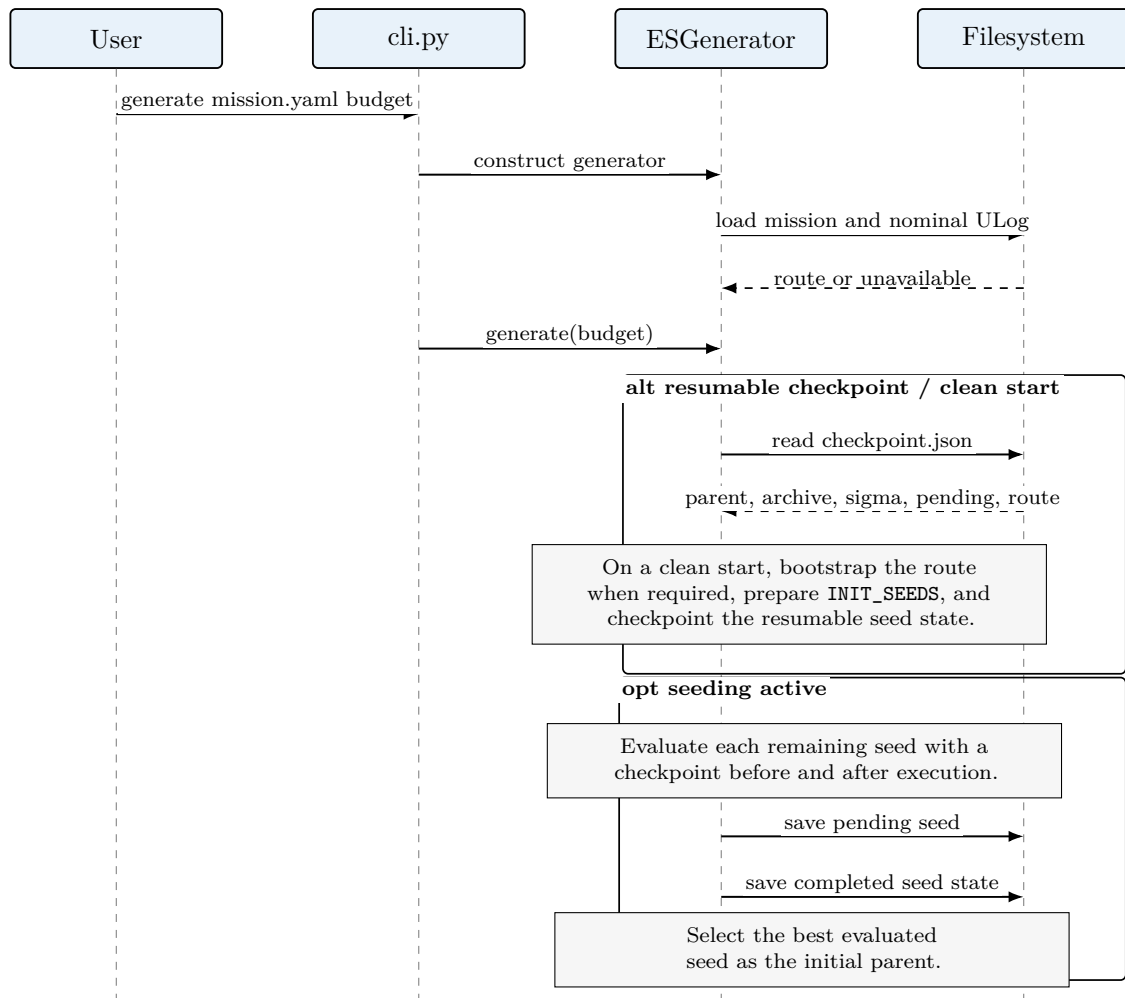


Figure 2.4: Runtime sequence, panel 1: invocation, route acquisition, checkpoint-aware initialization, and resumable seeding.

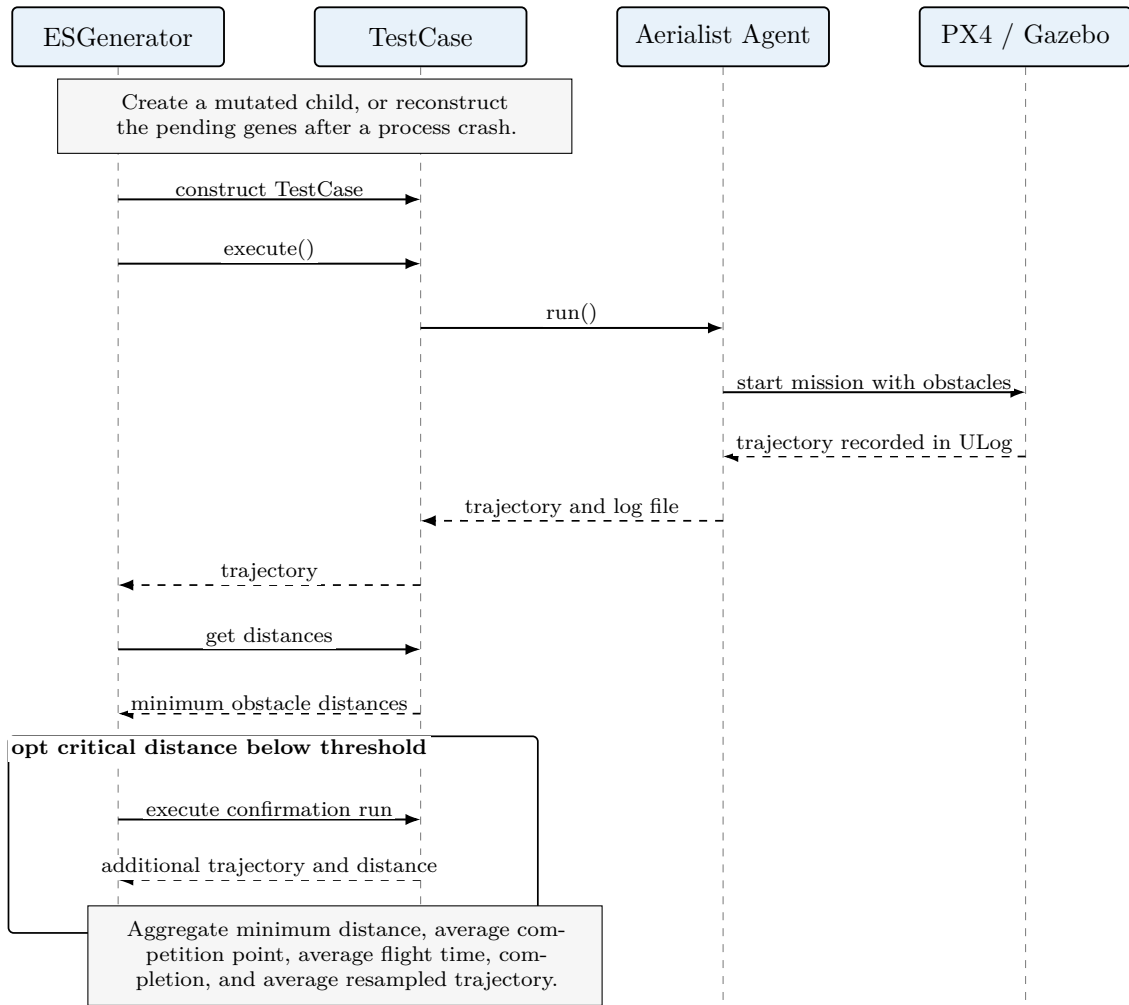


Figure 2.5: Runtime sequence, panel 2: candidate execution through Aerialist, telemetry return, distance measurement, and optional anti-noise confirmation.

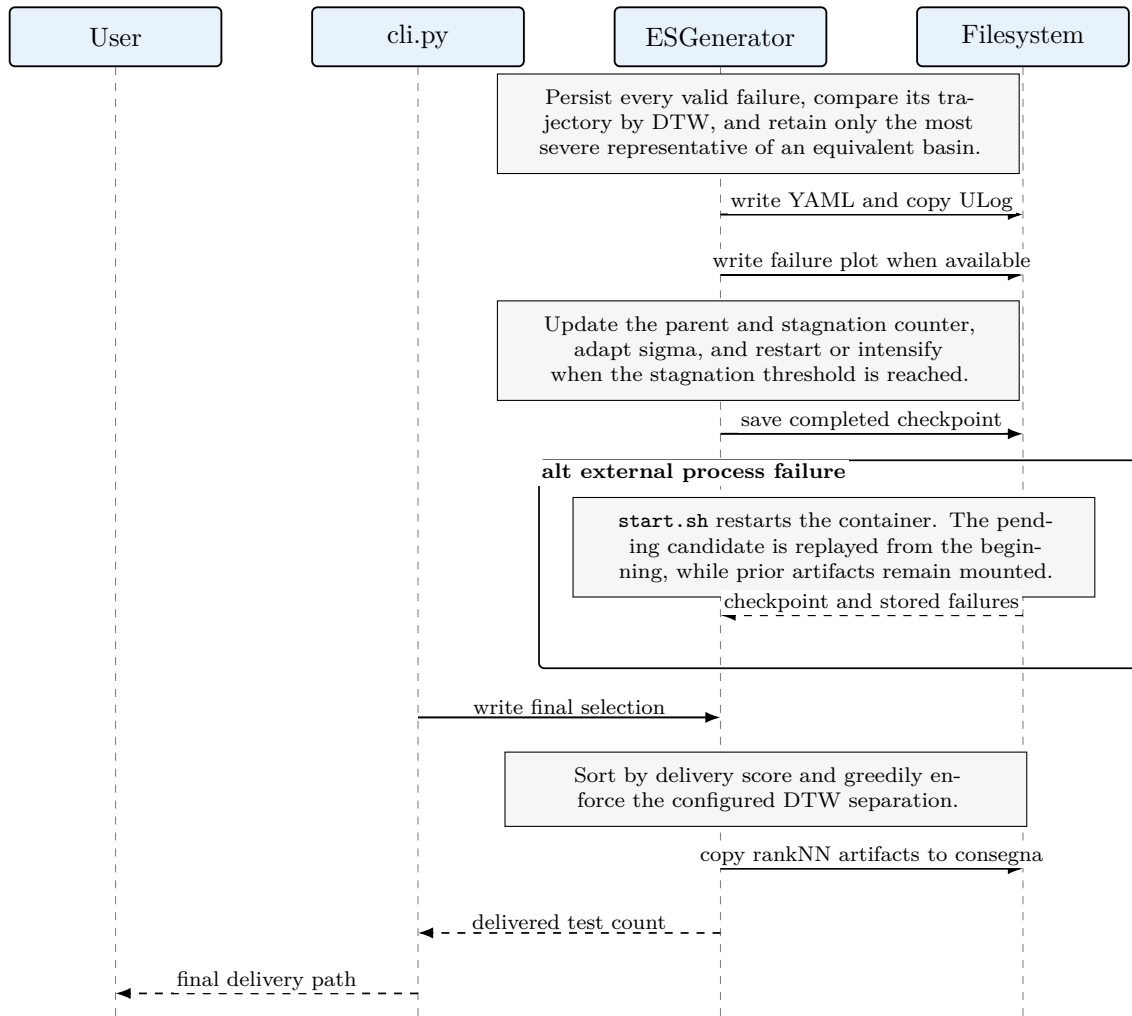


Figure 2.6: Runtime sequence, panel 3: persistent failure handling, archive update, recovery semantics, and final ranked delivery. Together with Figures 2.4 and 2.5, this is synchronized with docs/uml/sequence_diagram.mmd.

2.7.2 Detailed Execution Flow

Figures 2.7, 2.8, and 2.9 provide the portrait document rendering of the repository flowchart. Together they cover clean initialization, checkpoint restoration, seeding, the complete evolutionary iteration, stagnation handling, and delivery construction.

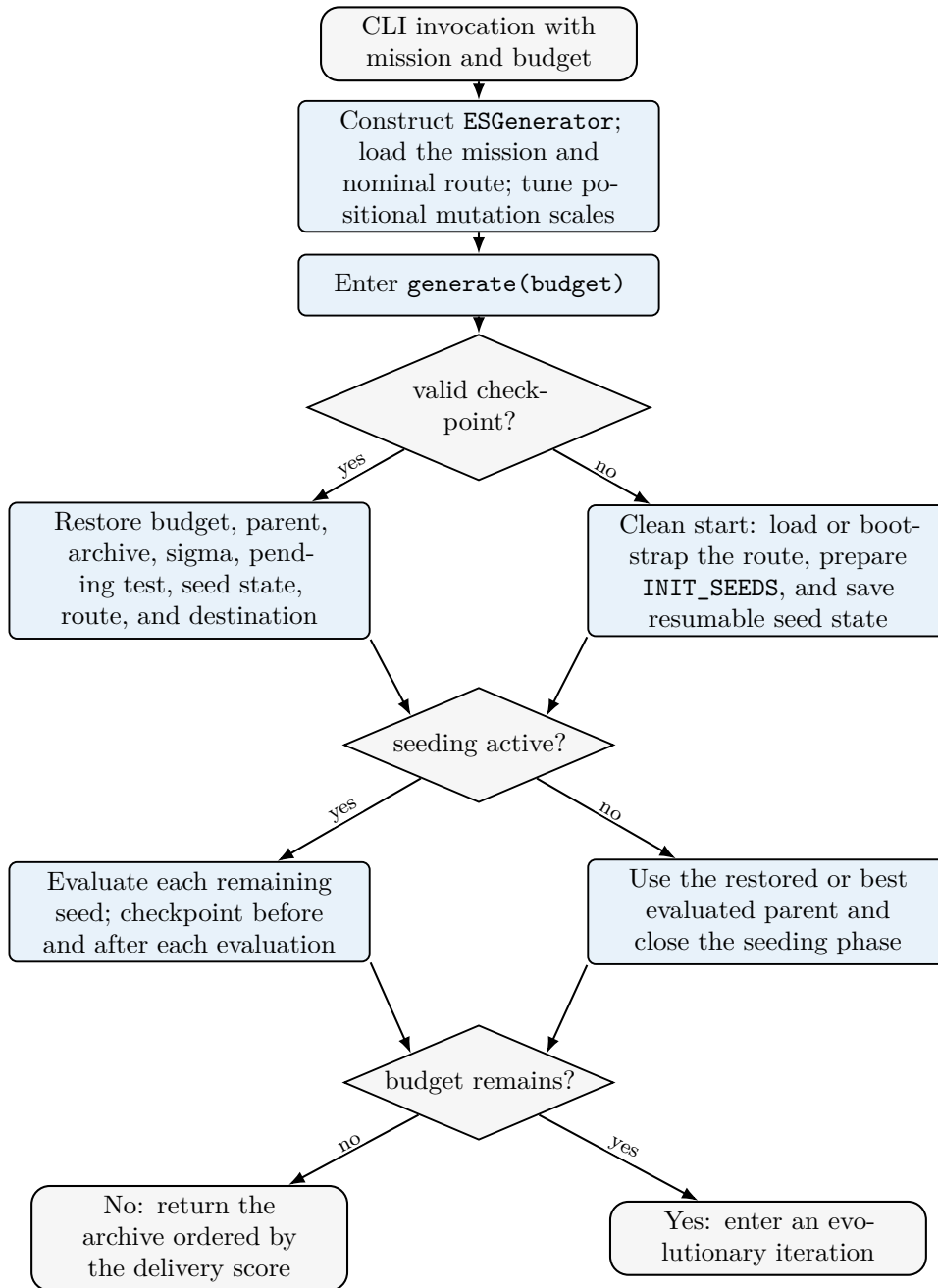


Figure 2.7: Execution-flow panel 1: construction, checkpoint restoration, route acquisition or bootstrap, resumable seeding, and loop entry.

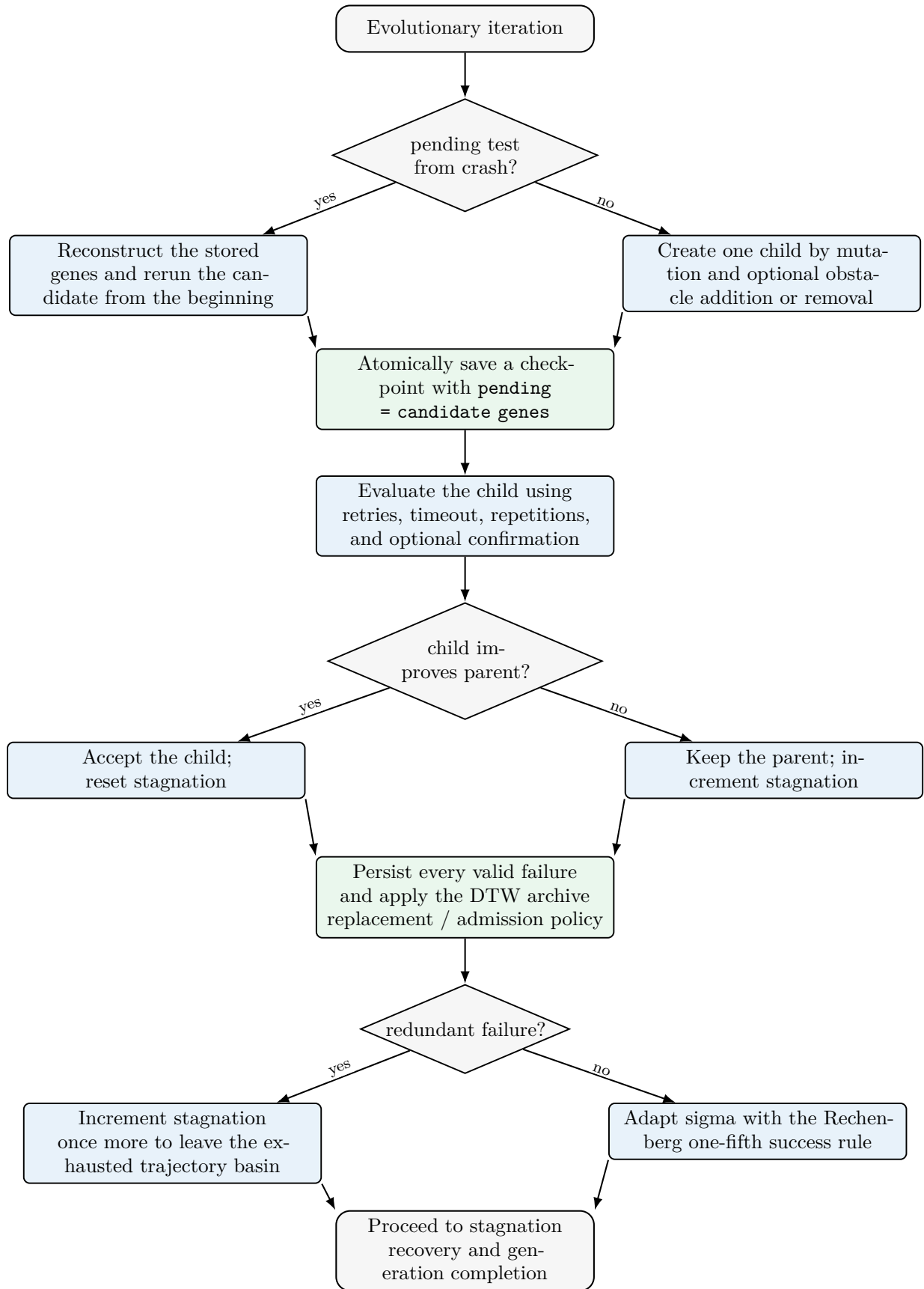


Figure 2.8: Execution-flow panel 2: candidate reconstruction or creation, evaluation, parent selection, persistent failure handling, archive update, and sigma adaptation.

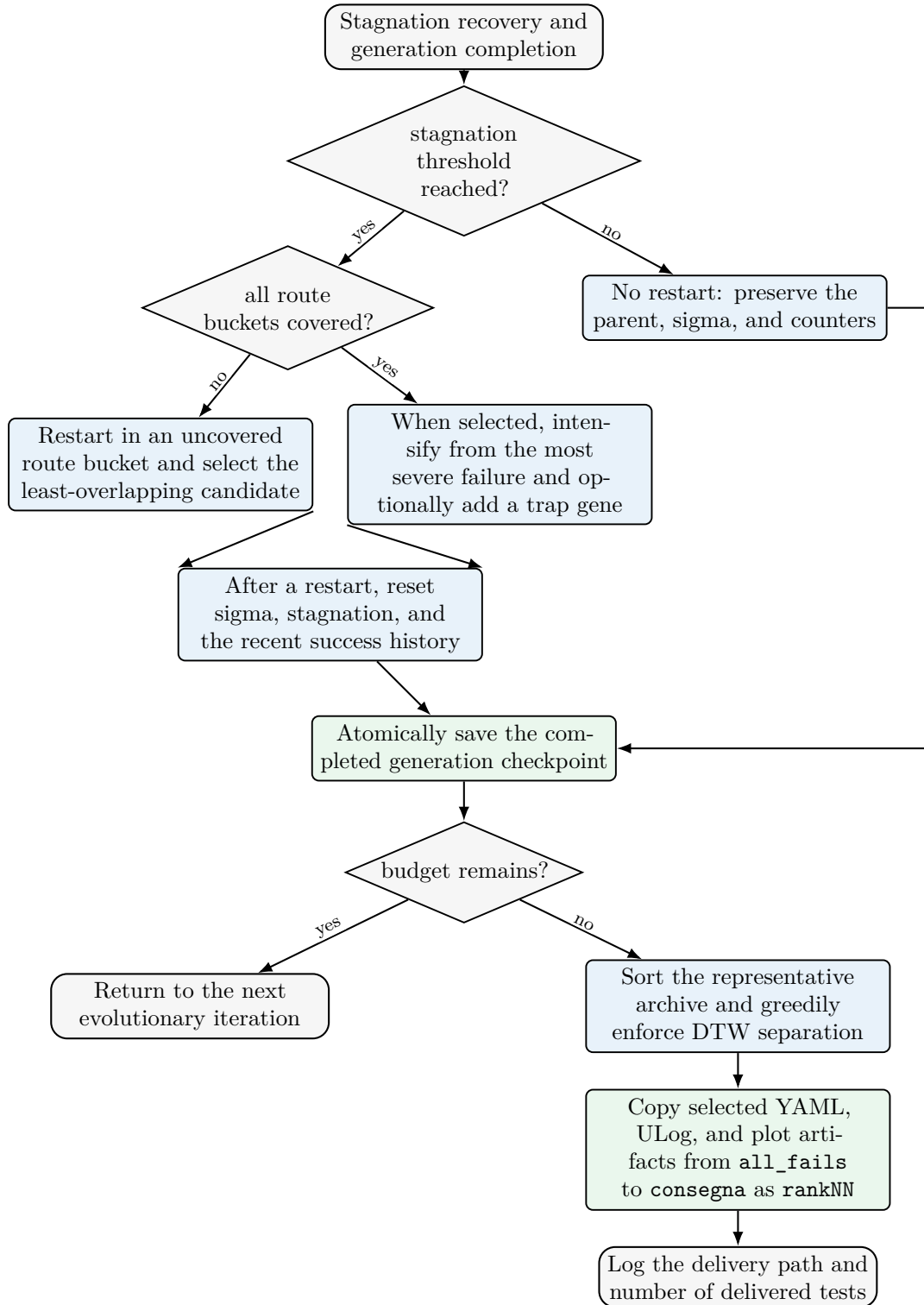


Figure 2.9: Execution-flow panel 3: stagnation recovery, restart policy, checkpoint completion, loop continuation, and final delivery. Together with Figures 2.7 and 2.8, this is synchronized with `docs/uml/execution_flow.mmd`.

2.7.3 Crash and Resume

Before every candidate evaluation, the controller records the candidate genes as `pending`. If the process terminates during the simulator call, the checkpoint still references that candidate. `start.sh` re-enters the same output directory, the generator restores the checkpoint, reconstructs

an **Individual** from the pending genes, and executes the full candidate again. Partial simulator state is never reused.

This design establishes the following recovery invariant:

At any restart point, all failures completed before the most recently pending candidate are persistent, and the next semantically valid action is to repeat that pending candidate from the beginning.

2.7.4 Final Delivery Construction

The controller sorts the archive by delivery score. It greedily scans the sorted list and retains a candidate only when its trajectory is sufficiently separated from every already retained trajectory. For each retained individual, the controller copies available `.yaml`, `.ulg`, and `_plot.png` files from `all_fails` to `consegna` using a rank-prefixed basename. A cleanup pass removes only delivery files whose base names are no longer expected.

2.8 Algorithm Design

2.8.1 Search Fitness

For an execution trajectory τ and obstacle set O , let $d(\tau, o)$ be the minimum trajectory-to-obstacle distance. The candidate fitness is

$$f(G) = \min_{o \in O} d(\tau, o).$$

When a candidate is executed multiple times, the implementation uses the minimum observed distance as fitness:

$$f(G) = \min_{k \in \{1, \dots, m\}} \min_{o \in O} d(\tau_k, o).$$

Selection is strict minimisation. The number of obstacles is deliberately excluded from fitness so that the search does not trade a less severe failure for simplicity prematurely.

2.8.2 Initialisation

In adaptive mode, obstacle centres are sampled around a randomly chosen point of the nominal route:

$$x = x_{\text{route}} + \varepsilon_x, \quad y = y_{\text{route}} + \varepsilon_y,$$

with Gaussian jitter followed by clamping and rotation-aware fitting. A configurable number of single-obstacle seeds is evaluated. The seed with minimum fitness becomes the first parent.

2.8.3 Gaussian Mutation

Each gene coordinate is mutated independently:

$$g'_i = \text{clamp} \left(g_i + \mathcal{N} \left(0, \sigma^2 s_i^2 \right), g_i^{\min}, g_i^{\max} \right),$$

where s_i is a dimension-specific step scale and σ is the global mutation scale. Position scales are derived from the route extent and bounded by configurable minimum and maximum values. After mutation, obstacle centres are corrected so that the complete rotated boxes remain legal.

The offspring operator may also:

- add an obstacle when fewer than three are present;
- remove a random obstacle when more than one is present;
- repair overlaps by retaining the first obstacle and discarding later overlapping obstacles.

2.8.4 Mutation-Scale Adaptation

The implementation applies Rechenberg’s 1/5 success rule over a bounded recent window. Let

$$p_s = \frac{1}{w} \sum_{j=1}^w I_j,$$

where $I_j = 1$ when the child strictly improved the parent and 0 otherwise. The scale update is

$$\sigma' = \begin{cases} \min(1.22\sigma, \sigma_{\max}), & p_s > 0.2, \\ \max(\sigma/1.22, \sigma_{\min}), & p_s \leq 0.2. \end{cases}$$

The scale is unchanged until the success-history window is full.

2.8.5 Trap-Gene Operator

When a new obstacle is added and a parent trajectory is available, the trap operator may replace pure random insertion.

For every trajectory point, the algorithm measures its distance to the existing obstacles and chooses the point closest to them. This point represents the gap through which the UAV escaped. A discrete local tangent estimates the direction of motion. The proposed obstacle is oriented approximately perpendicular to this direction. Its long side is proportional to the measured gap, bounded by the legal dimension range and an explicit cap. Gaussian position jitter prevents identical insertions. The candidate is accepted only if it does not overlap existing obstacles.

2.8.6 Stagnation and Restart

The stagnation counter increases after every non-improving child and receives an additional increment for a redundant failure. Once the threshold is reached, the generator replaces the parent and resets the mutation scale.

A diversified restart partitions the legal y interval into buckets. It prefers an uncovered bucket and generates several candidate genes, selecting the one with minimum maximum IoU against archived obstacles. When every bucket is represented, the generator may intensify around the most severe archived failure by copying its genes and, if possible, adding a trap gene.

2.8.7 Failure Point and Delivery Score

For a simulation with minimum distance d , the point value is

$$p(d) = \begin{cases} 5, & d < 0.25, \\ 2, & 0.25 \leq d < 1.0, \\ 1, & 1.0 \leq d < 1.5, \\ 0, & d \geq 1.5. \end{cases}$$

For a test case t , the delivery score is

$$S(t) = \frac{10\bar{p}(t)}{n_{\text{obs}}(t)^2 \bar{T}(t)},$$

where $\bar{T}$ is clamped away from zero to avoid numerical blow-up. This score is used only for archive eviction and final ordering.

2.8.8 Trajectory Representation and DTW

A successful trajectory is resampled to a fixed number N of (x, y) points. When repeated executions are available, corresponding samples are averaged.

For trajectories $A = (a_1, \dots, a_n)$ and $B = (b_1, \dots, b_m)$, the dynamic-programming recurrence is

$$D_{i,j} = \|a_i - b_j\|_2 + \min\{D_{i-1,j}, D_{i,j-1}, D_{i-1,j-1}\},$$

with $D_{0,0} = 0$ and unreachable boundary cells set to infinity. The final value is normalised by the backtracked alignment length, yielding a mean deviation in metres.

Two failures belong to the same trajectory basin when their DTW distance is below `DTW_THRESHOLD`. When trajectory data are unavailable, geometric IoU is used as a fallback.

2.8.9 Archive Policy

For every failure $f(G) < 1.5$:

1. save its artifacts immediately to `all_fails`;
2. search the archive for the closest trajectory basin;
3. if a duplicate basin exists, retain only the candidate with lower minimum distance as the in-memory representative;
4. if no duplicate exists and capacity is available, append it;
5. if capacity is full, replace the lowest delivery-score entry only when the new candidate has a higher score.

This separates complete evidence preservation from bounded evaluator-facing selection.

2.9 Error Handling and Fault Tolerance

Failure mode	Local handling	Persistent effect
Simulation timeout	Cancel alarm, log warning, retry if budget and retry count permit.	Attempt remains counted; checkpoint still protects the candidate.
Aerialist execution exception	Log compact diagnostic and retry.	Attempt remains counted.
Container/process crash	External wrapper restarts container.	Last valid checkpoint and previously dumped failures remain.
Interrupted candidate	Reconstruct from pending genes and repeat from start.	No partial simulation is treated as a valid result.
Unreadable checkpoint	Log warning and use documented clean-start behaviour.	Existing failure files are not deleted.
Missing nominal ULog	Attempt route bootstrap; otherwise fixed sampling region.	Route fallback is reflected in logs and checkpoint.
Missing trajectory metadata	Use neutral or geometric fallbacks for completion, time, or similarity.	Candidate may remain usable.
Plotting failure	Log warning and continue.	YAML and ULog remain deliverable.
File-copy failure	Log rank-specific warning and continue with remaining ranks.	Source failure files remain in all_fails .
User interruption	Launcher disables restart loop and terminates.	Checkpoint and failure history remain available.

2.10 Deployment View

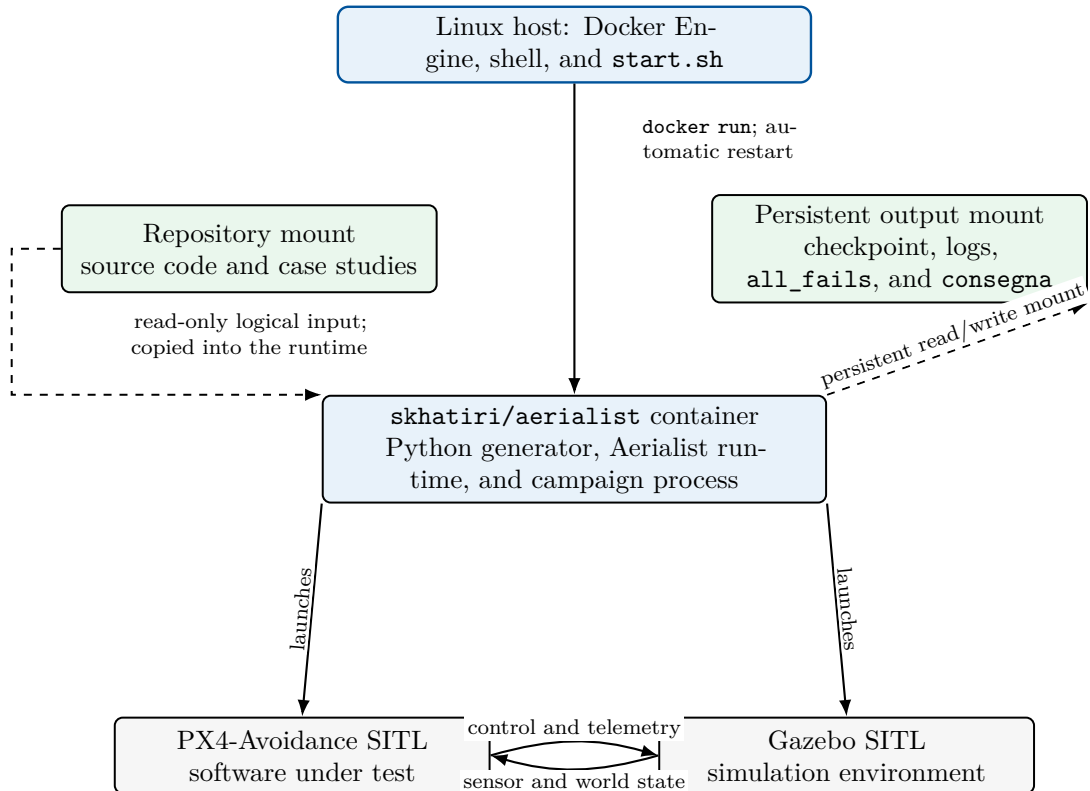


Figure 2.10: Reference Docker deployment and the two host-mounted persistence boundaries.

2.11 Architectural Styles and Patterns

2.11.1 Modular / Layered Organisation

The code separates interface, orchestration, domain model, deterministic domain services, infrastructure adapter, and presentation concerns. The separation is pragmatic rather than enforced by a framework, but dependencies predominantly point inward toward simple model and geometry logic.

2.11.2 Strategy Variation through Inheritance

`RandomGenerator` is substitutable for `ESGenerator` at the CLI boundary. The candidate-generation and restart methods are variation points, whereas the expensive common infrastructure is inherited. This resembles the Strategy and Template Method patterns: `generate()` defines the invariant process, and selected protected methods define the varying policy.

2.11.3 Adapter

`TestCase` adapts the Aerialist execution API to the generator’s needs. It hides agent selection and exposes a minimal interface: `execute`, `obtain distances`, and `save YAML`.

2.11.4 Checkpoint / Memento

The checkpoint is an explicit serialisable snapshot of the generator’s logical state. It resembles the Memento pattern while avoiding opaque object serialisation. The generator remains responsible

for producing and restoring the memento.

2.11.5 Repository-Like Failure Store

The filesystem-backed `all_fails` directory provides durable append-style storage, while the in-memory archive provides a bounded set of representatives. The distinction avoids coupling evidence retention to evaluator capacity.

2.11.6 Functional Core, Imperative Shell

Geometry, scoring, DTW, mutation, and serialisation are implemented as testable functions. The stateful `ESGenerator` forms an imperative shell that sequences simulator and filesystem effects. This separation improves testability and reduces the scope of failure-prone code.

2.12 Design Decisions and Trade-Offs

Decision	Benefit	Trade-off
Minimum distance as search fitness	Continuous gradient-like signal; directly targets safety severity.	Ignores obstacle simplicity during search; handled later by ranking.
One parent and one child	Small state, simple checkpointing, efficient under expensive simulations.	Less population diversity than larger evolutionary algorithms.
Adaptive route sampling	Concentrates evaluations in mission-relevant regions.	Requires a nominal log or one budget-consuming bootstrap.
Route-dependent mutation scale	Prevents position mutation from degenerating into global random jumps.	Introduces campaign-specific mutable geometry state.
DTW on resampled trajectories	Behavioural rather than purely geometric diversity; bounded cost.	$O(N^2)$ pairwise comparison and dependence on usable trajectories.
Immediate failure dump	Preserves evidence before the next simulator crash.	Produces more files than the final delivery requires.
Minimum observed distance over repetitions	Protects discovery of severe stochastic outcomes.	Can favour rare events; confirmation and average points partially mitigate this.
Atomic JSON checkpoint	Human-readable, portable, robust, easy to inspect.	Requires explicit serialisation and excludes live Aerialist objects.
Inheritance for baseline	Maximises infrastructure equivalence and reduces duplication.	Baseline still traverses the shared orchestration structure, even where some ES state is irrelevant.
External restart loop	Recovers even when Python cannot catch the failure.	Requires Docker and shell-level coordination.

User Interface Design

3.1 Interaction Model

The application exposes a command-line interface because its primary users are evaluators and developers running automated simulation campaigns. There is no interactive form or graphical application. Configuration is split between required positional arguments and optional environment variables.

3.2 Primary Commands

3.2.1 Reference Launcher

```
./start.sh <budget> <mission> [headless]

# Example: evolutionary campaign, 100 attempts, mission 1
./start.sh 100 mission1

# Random-search baseline
GENERATOR=random ./start.sh 100 mission1
```

3.2.2 Direct Generator Invocation

```
python3 cli.py generate case_studies/mission1.yaml 100
```

3.2.3 Self-Contained Container Build

```
docker build -t uav-es-generator .
docker run uav-es-generator \
  python3 cli.py generate case_studies/mission1.yaml 100
```

3.3 Environment Configuration

Variable	Default	Effect
GENERATOR	es	Selects evolutionary or random generator.
SEED	unset	Seeds Python's pseudo-random generator.

Variable	Default	Effect
ADAPTIVE	1	Enables route-guided sampling and route bootstrap.
N_RUNS	1	Base number of executions per candidate.
TEST_TIMEOUT	500	Maximum seconds per simulation attempt.
SIM_RETRIES	3	Maximum attempts within one <code>_run_once</code> call, subject to budget.
INIT_SEEDS	5	Number of initial evolutionary candidates.
STAGNATION_LIMIT	15	Non-improving generations before restart.
ARCHIVE_CAP	20	Maximum representative failures.
DTW_THRESHOLD	1.5	Mean trajectory deviation below which failures are considered equivalent.
TRAJ_N	100	Resampled points per trajectory.
INTENSIFY_PROB	0.4	Probability of intensifying after full spatial coverage.
TRAP_PROB	0.7	Probability of targeted rather than random obstacle insertion.
CONFIRM_BELOW	1.0	Distance threshold triggering confirmation.
CONFIRM_EXTRA_RUNS	1	Additional executions for a critical candidate.
MAX_RESTARTS	12	Launcher-level container restart limit.

3.4 Console Feedback

At campaign start, `start.sh` prints the selected generator, mission, budget, seed, initialisation settings, diversity threshold, archive capacity, and expected output location. During execution, the Python logger reports:

- obstacle parameters for each candidate;
- global attempt number and budget;
- minimum distance, obstacle count, completion status, and duration;
- confirmation, replacement, redundancy, restart, and resume events;
- final delivery count and directory.

Detailed exceptions and debug information are written to `debug.txt`. This dual-channel design keeps terminal output readable while preserving diagnostics.

3.5 Output Interface

The evaluator-facing interface is the `consegna` directory. Files are ordered lexicographically and semantically through the `rankNN` prefix. `rank01` is the first test to evaluate. YAML is the required test description; ULog and PNG files support inspection and presentation but do not change test semantics.

Requirements Traceability

4.1 Traceability Method

The following matrix maps every functional requirement and every non-functional requirement to design components and concrete implementation locations. A range is used only where each listed requirement is realised by the same design elements.

Requirement	Design element	Implementation evidence
FR-01–FR-02	CLI component	<code>cli.py: arg_parse()</code> , main exception boundary
FR-03	Generator-selection strategy	<code>cli.py: GENERATOR; random_generator.py</code>
FR-04–FR-05	Configuration component	<code>config.py</code> ; environment propagation in <code>start.sh</code>
FR-06	Simulation adapter	<code>ESGenerator.__init__</code> ; <code>TestCase.__init__</code> deep copy and obstacle replacement
FR-07	Route service	<code>geometry._load_route()</code>
FR-08	Route-bootstrap workflow	<code>ESGenerator._bootstrap_route_if_needed()</code>
FR-09	Route-dependent mutation	<code>geometry._set_step_scale()</code>
FR-10	Graceful route fallback	<code>geometry._load_route()</code> ; bootstrap fallback logging
FR-11	Candidate data model	<code>models.Individual</code> ; gene representation
FR-12–FR-13	Geometry validity service	<code>config.GENE_LOW/HIGH</code> ; <code>geometry._fit_center()</code>
FR-14	Overlap validation	<code>_boxes_overlap()</code> , <code>_overlaps_any()</code> , <code>_repair_overlaps()</code>
FR-15	Evolutionary offspring policy	<code>ESGenerator._make_child()</code>
FR-16	Trap-gene policy	<code>geometry._trap_gene()</code> and <code>TRAP_PROB</code>
FR-17	Random strategy	<code>RandomGenerator._make_child()</code> and <code>_restart_individual()</code>
FR-18	Simulation adapter	<code>TestCase.execute()</code> and Aerialist agents
FR-19–FR-20	Budget controller	<code>ESGenerator._run_once()</code> , bootstrap loop, <code>generate()</code> loop guards
FR-21–FR-22	Timeout and retry controller	SIGALRM handler; <code>_run_once()</code> ; <code>SIM_RETRIES</code>
FR-23–FR-24	Measurement and fitness service	<code>TestCase.get_distances()</code> ; <code>ESGenerator._evaluate()</code>
FR-25	Completion measurement	<code>geometry._reached_destination()</code> ; completion aggregation
FR-26	Evaluation aggregation	duration extraction, <code>_point()</code> , <code>_resample_xy()</code> in <code>_evaluate()</code>

Requirement	Design element	Implementation evidence
FR-27	Anti-noise confirmation	CONFIRM_BELOW, CONFIRM_EXTRA_RUNS, extended run loop
FR-28	Seed initialisation state machine	<code>generate()</code> ; <code>_eval_seed()</code> and resumable seed fields
FR-29	(1 + 1) selection	strict comparison in <code>generate()</code>
FR-30	Step-size adaptation	<code>ESGenerator._adapt_sigma()</code>
FR-31	Stagnation controller	stagnation updates and threshold check in <code>generate()</code>
FR-32	Diversified restart	<code>_restart_individual()</code> , y buckets, RESTART_CANDIDATES, IoU novelty
FR-33	Intensification	<code>_intensify_individual()</code> and INTENSIFY_PROB
FR-34	Redundancy pressure	ARCH_REDUNDANT branch in <code>generate()</code>
FR-35	Failure predicate	SOFT_FAIL; <code>_maybe_archive()</code>
FR-36–FR-37	Persistent failure store	<code>ESGenerator._dump_fail()</code>
FR-38–FR-39	Trajectory archive	<code>_dup_of()</code> , <code>_traj_gap()</code> , <code>_replace_in_archive()</code>
FR-40	Bounded archive eviction	ARCHIVE_CAP; lowest <code>_ranking_score</code> victim
FR-41	Delivery diversity selector	<code>ESGenerator._delivery_selection()</code>
FR-42	Ranking formula	<code>geometry._ranking_score()</code>
FR-43–FR-44	Idempotent rank writer	<code>write_final_selection()</code> ; rank naming and delivery-only cleanup
FR-45–FR-46	Atomic checkpoint writer	<code>save_checkpoint()</code> ; temporary file and <code>os.replace()</code>
FR-47	Checkpoint schema	<code>save_checkpoint()</code> ; model serialisation helpers
FR-48–FR-49	Restore and pending replay	<code>_load_checkpoint()</code> ; pending branches in seed and main loops
FR-50	External recovery wrapper	restart loop, trap, persistent mounts, and ownership handling in <code>start.sh</code>
FR-51	Logging subsystem	<code>config_loggers()</code> , generator logs, persistent <code>debug.txt</code>
FR-52	Completion feedback	<code>write_final_selection()</code> log and final lines in <code>start.sh</code>
FR-53	Plot component	<code>plot_official.plot_one()</code> and <code>ESGenerator._plot_fail()</code>
FR-54	Non-critical plotting	exception boundary in <code>_plot_fail()</code>
FR-55	Optional visual mode	HEADLESS and X11 handling in <code>start.sh</code>
NFR-01	Geometry correctness	<code>geometry.py</code> ; validation and repair functions
NFR-02	Budget safety	pre-attempt increments and all loop guards in <code>es_generator.py</code>
NFR-03	Recoverability	checkpoint protocol, immediate failure dump, and launcher restart
NFR-04	Reliability	local exception boundaries in route loading, simulation, checkpoint, plotting, and delivery
NFR-05	Reproducibility	SEED handling in <code>config.py</code> ; seed-labelled run directory
NFR-06	Bounded performance	TRAJ_N, ARCHIVE_CAP, resampled DTW
NFR-07	Portability	Dockerfile and Docker-based <code>start.sh</code>
NFR-08	Maintainability	explicit module decomposition in Figure 2.2
NFR-09	Extensibility	inheritance-based generator variation point
NFR-10	Observability	persistent and console log design

Requirement	Design element	Implementation evidence
NFR-11	Usability	one-command launcher and final delivery path
NFR-12	Traceability	this chapter and code/UML documentation
NFR-13	Data preservation	copy-only delivery and <code>all_fails</code> non-deletion invariant
NFR-14	Graceful degradation	neutral defaults and fallback paths in geometry, scoring, checkpoint, and plotting
NFR-15	Testability	functional geometry/model modules separated from <code>TestCase.execute()</code>

Implementation, Integration, and Test Plan

5.1 Implementation Order

Although the final system is complete, the following dependency-aware order describes the intended implementation process:

1. define official constants and parameter loading in `config.py`;
2. implement the `Individual` model and JSON-safe serialisation;
3. implement obstacle conversion, legal fitting, overlap detection, sampling, mutation, and scoring in `geometry.py`;
4. implement the Aerialist `TestCase` adapter;
5. implement one-candidate execution, timeout, retries, and budget accounting;
6. implement initialisation and the basic $(1 + 1)$ selection loop;
7. add checkpoint save/restore and pending-candidate replay;
8. add failure persistence, archive capacity, deduplication, and delivery ranking;
9. add route adaptation, restart, intensification, trap genes, and confirmation runs;
10. add the random baseline, plotting, Docker build, and restart launcher;
11. complete documentation, experiments, and final consistency checks.

5.2 Integration Strategy

Integration proceeds from deterministic units toward external simulation:

1. **Model–geometry integration:** verify gene conversion, legal fitting, overlap handling, and serialisation without Aerialist execution.
2. **Geometry–generator integration:** exercise child generation, selection, sigma adaptation, restart, archive, and delivery using synthetic individuals.
3. **Generator–filesystem integration:** verify atomic checkpoints, relative artifact references, idempotent delivery, and cleanup boundaries in temporary directories.
4. **Generator–Aerialist integration:** execute one known mission with a minimal budget and confirm trajectory, distance, duration, and log extraction.
5. **Container integration:** run the same minimal campaign through `start.sh`, including mounted outputs and ownership restoration.

6. **Recovery integration:** interrupt a campaign during execution and verify pending replay, monotonic budget accounting, and archive preservation.
7. **Campaign validation:** execute full-budget evolutionary and random campaigns and inspect delivered artifacts and experimental summaries.

5.3 Test Levels

5.3.1 Unit Tests

Unit tests should cover:

- clamping and rotation-aware legal fitting;
- rotated-box overlap for separated, touching, and intersecting cases;
- non-overlapping sampling and overlap repair;
- point bands and delivery ranking;
- trajectory resampling and DTW identity/symmetry expectations;
- individual serialisation round-trip, including unavailable values;
- mutation-scale bounds and success-rule transitions;
- duplicate archive replacement and archive-cap eviction;
- final greedy delivery separation.

5.3.2 Integration Tests

Integration tests should cover:

- CLI parsing and generator selection;
- mission loading and route fallback;
- persistent checkpoint creation and restoration;
- failure dump and delivery copy using temporary files;
- ULog plotting with at least one known artifact;
- Docker execution with a low budget;
- launcher restart with a deliberately terminated container.

5.3.3 System Tests

System tests execute full campaigns and verify:

- budget exhaustion without excess attempts;
- legal YAML outputs;
- complete `all_fails` evidence and bounded `consegna`;
- rank ordering by the documented score;
- meaningful distinction between evolutionary and random search results;
- successful execution on Missions 1, 2, and 3.

5.3.4 Regression Tests

Every modification to geometry, scoring, archive, checkpoint schema, or output naming should trigger regression tests. Backward-compatible checkpoint fallbacks should be maintained when field names change, or a migration policy should be stated explicitly.

5.4 Test Oracles

Subject	Oracle
Obstacle validity	Numerical bounds, $z = 0$, cardinality, rotation-aware footprint containment, and pairwise non-overlap.
Fitness	Minimum value returned by trajectory-to-obstacle distance computations.
Point score	Exact piecewise thresholds at 0.25, 1.0, and 1.5 metres.
Ranking	Independent calculation of $10\bar{p}/(n_{\text{obs}}^2 \bar{T})$.
DTW duplicate	Identical trajectories yield zero; known shifted trajectories exceed a selected threshold.
Budget	Number of attempt log entries and checkpoint counter never exceeds declared budget.
Checkpoint recovery	Restored logical state equals the state saved before termination, excluding non-serialised live objects.
Delivery	Rank files correspond to retained archive entries, are score ordered, and are separated according to the configured trajectory rule.
Data preservation	Hashes and file counts in <code>all_fails</code> remain unchanged after rebuilding delivery.

References

- [1] R. Capone and G. Cattaneo, *Requirements Analysis and Specification Document – Search-Based Test Generation for Autonomous UAV Obstacle Avoidance Using a (1 + 1)-Evolution Strategy*, version 1.0, 17 July 2026.
- [2] M. Camilli, *Software Engineering (for Automation) – Project Guideline*, Politecnico di Milano, A.Y. 2024–2025.
- [3] R. Capone and G. Cattaneo, *Search-Based Test Generation for Autonomous UAV Obstacle Avoidance Using a (1 + 1)-Evolution Strategy*, Software Engineering Project Repository, 2026. Available: https://github.com/gulicatta/SE4A_RobertoCapone_GuglielmoCattaneo.
- [4] UAV Testing Competition, *Official Repository and Competition Documentation*. Available: <https://github.com/skhatiri/UAV-Testing-Competition>.
- [5] S. Khatiri et al., “SBFT Tool Competition 2024 – CPS-UAV Test Case Generation Track,” in *International Workshop on Search-Based and Fuzz Testing*, 2024.
- [6] S. Khatiri, T. Zohdinasab, P. Saurabh, D. Humeniuk, and S. Panichella, “ICST Tool Competition 2025 – UAV Testing Track,” in *IEEE International Conference on Software Testing, Verification and Validation*, 2025.
- [7] S. Khatiri, S. Panichella, and P. Tonella, “Simulation-based Testing of Unmanned Aerial Vehicles with Aerialist,” in *International Conference on Software Engineering*, 2024.
- [8] S. Khatiri, S. Panichella, and P. Tonella, “Simulation-based Test Case Generation for Unmanned Aerial Vehicles in the Neighborhood of Real Flights,” in *IEEE International Conference on Software Testing, Verification and Validation*, 2023.
- [9] I. Rechenberg, *Evolutionsstrategie: Optimierung technischer Systeme nach Prinzipien der biologischen Evolution*. Frommann-Holzboog, 1973.

Implementation File Catalogue

File	Architectural role
<code>cli.py</code>	Entry point, generator selection, argument parsing, logging, top-level error boundary.
<code>config.py</code>	Official bounds and environment-configurable parameters.
<code>models.py</code>	Individual data model and checkpoint serialisation.
<code>geometry.py</code>	Geometry, route, sampling, mutation, scoring, DTW, trap, and repair functions.
<code>testcase.py</code>	Aerialist execution adapter.
<code>es_generator.py</code>	Evaluation, checkpoint, (1 + 1)-ES, archive, and delivery orchestration.
<code>random_generator.py</code>	Independent random candidate policy using shared infrastructure.
<code>plot_official.py</code>	Failure visualisation from YAML and ULog artifacts.
<code>start.sh</code>	Docker launcher, persistent mounts, automatic restart, and ownership restoration.
<code>Dockerfile</code>	Self-contained image definition.
<code>requirements.txt</code>	Additional Python dependencies not provided by the base image.
<code>case_studies/</code>	Input missions and associated resources.
<code>generated_tests/</code>	Campaign checkpoints, logs, complete failures, and ranked delivery suites.
<code>docs/uml/</code>	Mermaid sources for class, sequence, and execution-flow diagrams.

Default Parameter Catalogue

Parameter	Default	Meaning
N_RUNS	1	Base executions per evaluation.
TEST_TIMEOUT	500 s	Per-attempt timeout.
SIM_RETRIES	3	Attempt limit inside one execution request.
CONFIRM_BELOW	1.0 m	Criticality threshold.
CONFIRM_EXTRA_RUNS	1	Additional confirmation execution.
ADAPTIVE	true	Route-guided placement.
ROUTE_JITTER	3.0 m	Sampling spread around route.
DEST_TOLERANCE	5.0 m	Destination reach radius.
COMPLETE_MIN_FRAC	0.5	Required reached fraction.
SIGMA_INIT	0.15	Initial mutation-scale fraction.
SIGMA_MIN	0.02	Lower mutation-scale bound.
SIGMA_MAX	0.4	Upper mutation-scale bound.
SOFT_FAIL	1.5 m	Failure threshold.
STAGNATION_LIMIT	15	Restart threshold.
INIT_SEEDS	5	Initial candidates.
RESTART_CANDIDATES	12	Candidates considered during diversified restart.
INTENSIFY_PROB	0.4	Intensification probability after coverage.
ARCHIVE_CAP	20	Representative failure capacity.
DUP_IOU	0.95	Geometric duplicate fallback threshold.
Y_BUCKET_SIZE	10 m	Spatial diversity bucket width.
DTW_THRESHOLD	1.5 m	Trajectory-basin threshold.
TRAJ_N	100	Resampled trajectory length.
POS_STEP_FRAC	0.6	Route-extent fraction for positional step.
POS_STEP_MIN	3.0 m	Minimum positional scale.
POS_STEP_MAX	8.0 m	Maximum positional scale.
TRAP_PROB	0.7	Targeted insertion probability.
TRAP_JITTER	1.0 m	Target-position jitter.
TRAP_SPAN_MULT	3.0	Gap-to-long-side multiplier.
TRAP_SIZE_FRAC	0.35	Long-side cap fraction.